

Einstieg in die Informatik aus HR mit Ziel IT-Projektleitung

Kernempfehlung

Für dein Ziel ist nicht die schnellste Route in die Softwareentwicklung entscheidend, sondern die schnellste Route zu **technischem Verständnis mit echter Praxisnähe**. Die klare Reihenfolge ist deshalb: **SQL zuerst**, dann **Python**, danach **Web- und API-Grundlagen**, anschließend **Git/GitHub**. **Java** ist nicht falsch, aber für deinen Einstieg unnötig schwer. **Angular** ist für den Anfang klar die falsche Wahl, weil es bereits HTML, CSS und JavaScript voraussetzt und zusätzlich in aktuelle Toolchains mit Node.js, TypeScript und RxJS eingebettet ist. ¹

Kurz und deutlich: **Bestes Einstiegsthema: SQL. Beste erste Sprache: Python. Beste Haupt-Hardware: dein Lenovo-Laptop. Raspberry Pi 3 B+: nur Nebenrolle, nicht Hauptgerät.** Dein Laptop ist für einen leichten Linux-Desktop, Browser, Python, SQLite und Git völlig ausreichend. Der Raspberry Pi 3 B+ hat dagegen nur 1 GB RAM und ist laut Hersteller zwar als Desktop oder Headless-Rechner nutzbar, aber für moderne, browserlastige Lernpfade spürbar limitierter. ²

Warum SQL der sinnvollste erste Schritt ist

SQL ist für deinen Hintergrund der beste Start, weil es die geringste Einstiegshürde mit dem höchsten Praxisnutzen kombiniert. SQLBolt beschreibt SQL ausdrücklich als Sprache, die technische **und nicht-technische** Nutzer verwenden können; relationale Tabellen werden dort direkt mit Kalkulationstabellen verglichen. Auch die Microsoft-Lerneinheit für T-SQL sagt, dass als Voraussetzung bereits Erfahrung mit Tabellen aus Spreadsheets hilfreich ist. Genau deshalb passt SQL so gut für jemanden aus HR, der wahrscheinlich bereits mit Listen, Auswertungen, Exporten und Berichten arbeitet. ³

Der zweite Grund ist psychologisch: Du bekommst mit SQL schnell sichtbare Ergebnisse. Schon einfache `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY` und `JOIN`-Abfragen erzeugen sofort einen Aha-Effekt. Das ist wichtig, weil Motivation am Anfang fast immer aus **sichtbarem Fortschritt** kommt, nicht aus Theorie. CS50 SQL startet ebenfalls genau dort und baut von Abfragen über Normalisierung und Beziehungen bis zu Views, Indizes und der Verbindung mit Python auf. ⁴

Ich würde dir deshalb **nicht** empfehlen, monatelang nur abstrakte Informatik zu lesen. Starte mit Daten, Tabellen und Abfragen. Das ist der geradeste Weg von „ich komme aus HR“ zu „ich verstehe, wie Systeme Informationen strukturieren“. Für dein Ziel als spätere Teilprojektleitung ist das wertvoller als ein früher Sprung in Objektorientierung oder Frontend-Frameworks. ⁵

Warum Python danach kommt und Java und Angular später

Nach den ersten SQL-Wochen sollte Python folgen, weil du damit den Schritt von **Daten lesen** zu **Logik formulieren und kleine Abläufe automatisieren** machst. CS50 Python ist explizit für Lernende mit oder ohne Vorerfahrung gedacht und betont außerdem, dass du außer einem Browser keine zusätzliche Software brauchst, wenn du erst einmal ohne lokale Einrichtung starten willst. Das ist für einen Fachfremden ideal, weil es Setup-Frust reduziert. Auch die kostenlose Microsoft-Serie „Python for

Beginners“ ist genau mit dem Problem gestartet, dass der größte Stolperstein am Anfang oft schlicht die Frage ist, **wo man überhaupt anfängt**. 6

Wichtig ist aber auch, was Python **nicht** ist: Die offizielle Python-Dokumentation ist sehr gut, aber sie sagt selbst, dass das Tutorial nicht für komplette Anfänger in Programmierung gedacht ist, sondern für Menschen, die Python lernen wollen und bereits etwas Programmierverständnis mitbringen. Das ist der Grund, warum ich dir für die erste Phase **keine** reine Doku-Strategie empfehle, sondern einen geführten Kurs plus kleine Übungen. 7

Java bleibt für später sinnvoll, vor allem wenn du später in einem eher klassischen Unternehmensumfeld arbeitest. Aktuelle GitHub-Daten zeigen Java weiterhin als zentrales Element moderner Projekt-Stacks, und GitHub nennt Java ausdrücklich einen verlässlichen Enterprise- und Backend-Workhorse. Der Haken liegt nicht in der Relevanz, sondern in der Einstiegslast: Die offiziellen Java-Lernmaterialien führen Neueinsteiger früh an Klassen, Vererbung, Interfaces und Packages heran. Das ist in Ordnung, aber als **erste** Programmiersprache für dein Ziel unnötig viel Formalismus. 8

Angular solltest du am Anfang komplett ignorieren. Die Angular-Dokumentation sagt klar, dass du für das Tutorial bereits Grundkenntnisse in HTML, CSS und JavaScript brauchst. MDN beschreibt JavaScript wiederum als dritte Schicht der klassischen Web-Grundlagen nach HTML und CSS. Außerdem zeigt Angulars Versions-Kompatibilitätsseite, dass das Framework in aktuelle Laufzeit- und Tooling-Anforderungen eingebunden ist. Übersetzt: **Erst Web verstehen, dann Frameworks**. Nicht umgekehrt. 9

Welche Lernplattformen für dich wirklich taugen

Für kostenlose Primärressourcen sind vor allem freeCodeCamp 10, Khan Academy 11, die CS50-Kurse der Harvard University 12, MDN von der Mozilla Foundation 13 und die Lernpfade von Microsoft 14 die beste Kombination. Der Grund ist einfach: Du brauchst als Fachfremder am Anfang **aktive** Lernformate. Der UNC Learning Center Guide empfiehlt verteiltes Lernen über kurze, intensive Einheiten und betont Self-Testing und aktives Arbeiten; Carnegie Mellon beschreibt aktive Lernstrategien ausdrücklich als Gegenmodell zum rein passiven Konsum. Genau deshalb sind interaktive Plattformen für dich als Hauptkanal besser als reine Videobibliotheken. 15

Meine klare Primärempfehlung ist daher: interaktive kostenlose Ressourcen zuerst, strukturierte OpenCourseWare danach, Videos nur ergänzend. Konkret heißt das: Khan Academy oder SQLBolt für den sehr sanften SQL-Start, dann CS50 SQL und CS50 Python für echte Tiefe, MDN für Web-Grundlagen und Microsoft Learn für SQL- oder GitHub-Module. freeCodeCamp ist komplett kostenlos, projektorientiert und zertifikatsbasiert; CS50 SQL und CS50 Python sind frei zugänglich; MDN stellt einen strukturierten Weg von „beginner“ zu „comfortable“ bereit; Microsoft Learn bietet modulare, schrittweise Lernpfade. 16

LinkedIn Learning ist für dich nur dann eine gute Startplattform, wenn dein Arbeitgeber den Zugang bezahlt oder du bereits über Premium Zugriff hast. Das Produkt ist sauber strukturiert und professionell aufbereitet, aber es setzt eine aktive Subscription voraus, verkauft keine einzelnen Kurse separat und bietet nur einen begrenzten Gratiszeitraum für berechnigte Nutzer. Für jemanden, der „am besten kostenlos“ lernen will, ist das nicht die beste erste Wahl. 17

Coursera ist besser, wenn du später einen formalen Abschluss oder eine Arbeitgebermarke auf dem Zertifikat willst. Die Plattform arbeitet mit Universitäten und Unternehmen zusammen und lässt heute meist den ersten Modulabschnitt vieler Kurse kostenlos testen; für volle Zugänge oder Zertifikate

brauchst du aber oft ein Probeabo, eine Subscription oder Financial Aid. Das ist gut, aber nicht der effizienteste Null-Euro-Start. ¹⁸

Udemy ist kein schlechter Ort, aber kein guter Kompass. Offiziell ist Udemy ein globaler Marktplatz mit regional variierenden Preisen; zusätzlich ist die Subscription nicht für alle Lernenden in gleicher Weise verfügbar. Übersetzt: Du kannst dort gute Kurse finden, aber du musst selbst sehr gut auswählen. Für Fachfremde ist das am Anfang meistens zu viel Auswahl und zu wenig Orientierung. ¹⁹

YouTube ist nützlich, aber nur als Zweitkanal. Offiziell kann dort jeder Playlists anlegen. Genau das macht YouTube stark für punktuelle Erklärungen und gleichzeitig schwach als Hauptcurriculum: Du findest schnell Hilfe zu Einzelproblemen, aber keine verlässliche Reihenfolge, keine saubere Progression und meistens auch keine eingebauten Übungen. Benutze YouTube daher nur für konkrete Lücken, nicht als Hauptfahrplan. ²⁰

Bücher sind für dich Begleitmaterial, nicht Startplattform. Das freie Pro-Git-Buch ist hervorragend, wenn du später Git wirklich verstehen willst. Die Python-Doku ist als Referenz stark, aber als Erstkontakt für absolute Anfänger ausdrücklich nicht gedacht. Mein Urteil ist deshalb eindeutig: **erst interaktiv lernen, dann Bücher und Doku als Nachschlagewerk benutzen**. ²¹

Ein realistischer Lernpfad für die ersten Monate

Die ersten Monate sollten nicht wie ein Informatikstudium aussehen. Sie sollten wie ein sauber aufgebautes Onboarding aussehen.

In den **ersten Wochen** startest du nur mit SQL. Nimm dafür zuerst einen extrem leichten Einstieg wie Khan Academy oder SQLBolt. Ziel ist nicht Zertifikatssammeln, sondern Sicherheit in den Kernbefehlen: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `COUNT`, `SUM`, `AVG`, `JOIN`. Sobald du diese Befehle ohne Abtippen aus einem Spickzettel formulieren kannst, wechselst du in einen tieferen Kurs wie CS50 SQL. Weil CS50 SQL mit SQLite für Portabilität beginnt und erst später PostgreSQL/MySQL streift, ist das genau die richtige Brücke vom Anfänger zur realen Datenbankwelt. ²²

Danach folgt **Python**. Dort reicht zunächst absolut das Fundament: Variablen, Bedingungen, Schleifen, Funktionen, Listen, Dictionaries, Dateien, Fehlerbehandlung. Nimm dafür CS50 Python oder die Microsoft-Serie für Anfänger. Wichtig ist, dass du am Anfang nicht in Bibliotheken, KI-Tools oder Webframeworks abdriftest. Wenn du mit Python ein CSV einlesen, filtern, gruppieren und eine kleine Textdatei oder Auswertung erzeugen kannst, bist du genau da, wo du hinmusst. ⁶

Erst danach kommen **Web-Grundlagen**. Nicht für eine Frontend-Karriere, sondern damit du Entwicklergespräche verstehst. MDN empfiehlt kompletten Anfängern ausdrücklich, mit den Getting-Started-Modulen zu beginnen, und ordnet JavaScript erst auf Basis von HTML und CSS ein. Für dich reicht hier ein solides Verständnis von HTML, CSS, JavaScript auf hoher Ebene, HTTP-Anfragen, APIs, JSON und dem Unterschied zwischen Frontend, Backend und Datenbank. Das macht dich in Projektbesprechungen sofort stärker. ²³

Danach nimmst du **Git und GitHub** dazu. Microsoft Learn nennt als Lernziele beim GitHub-Einstieg explizit Repositories, GitHub Flow, Branches, Commits, Pull Requests sowie kollaborative Elemente wie Issues und Discussions. GitHub Projects kann Aufgaben in Sub-Issues zerlegen, Fortschritt verfolgen und Roadmaps visualisieren. Für dein mittelfristiges Ziel in Richtung Teilprojektleitung ist das Gold wert, weil du damit nicht nur Technik verstehst, sondern gleichzeitig Denkweisen aus der Zusammenarbeit in Softwareteams lernst. ²⁴

Wenn du **nur fünf kostenlose Ressourcen** sofort öffnen willst, nimm diese:

1. **Khan Academy: Intro to SQL** – sehr sanfter Einstieg, gut für absolute Anfänger. ²⁵
2. **SQLBolt** – direkt im Browser, sehr schnell, sehr wenig Reibung. ²⁶
3. **CS50 SQL** – tiefer, praxisnäher, mit sauberem Aufbau von SQLite Richtung größere Datenbanksysteme. ²⁷
4. **CS50 Python** – stark geführt, praxisnah, browserbasiert möglich. ²⁸
5. **MDN Learn Web Development** – beste kostenlose Struktur für Web-Grundlagen ohne Framework-Hype. ²⁹

Bonus danach: Introduction to GitHub auf Microsoft Learn. Das lohnt sich besonders früh, wenn du dein Lernen sauber organisieren willst. ³⁰

CS50x selbst ist stark, aber ich würde es **nicht** als allerersten Schritt wählen. Die offizielle Kursseite spricht von elf Wochen Material plus Problem Sets und Final Project. Das ist exzellent als zweite Runde, wenn du bereits erste Sicherheit in SQL und Python hast; für einen HR-Quereinsteiger mit Motivationsrisiko ist es als ganz erster Brocken eher zu schwer. ³¹

Motivation und Dranbleiben

Wenn du dauerhaft dranbleiben willst, musst du dein Lernen wie ein Trainingsprogramm behandeln, nicht wie einen Motivationsschub. Das UNC Learning Center empfiehlt verteiltes Lernen über mehrere kurze Sessions statt Marathonblöcke und nennt 30- bis 45-minütige intensive Einheiten ausdrücklich wirksam. Außerdem wird dort Self-Testing als aktive Strategie hervorgehoben. Praktisch heißt das für dich: **vier Einheiten pro Woche, lieber kurz und regelmäßig als selten und heroisch.** ³²

Du brauchst außerdem sichtbare Ziele. UNC empfiehlt klar formulierte SMART-Ziele; die American Psychological Association verweist darauf, dass häufiges Fortschrittsmonitoring die Wahrscheinlichkeit erhöht, ein Ziel zu erreichen. Übersetzt: Nicht „ich will Informatik können“, sondern zum Beispiel „bis Sonntag zehn saubere SELECT-Abfragen mit GROUP BY schreiben“ oder „diese Woche zwei Python-Funktionen bauen, die CSV-Daten einlesen“. ³³

Ein dritter Punkt ist Reflexion statt bloßes Abarbeiten. Das UNC-Material zu Metakognition beschreibt genau den Effekt, dass selbstreflektierende Fragen helfen, den eigenen Lernstand, funktionierende Methoden und Lücken zu erkennen. Mach deshalb jede Woche eine Mini-Retrospektive mit drei Fragen: **Was habe ich verstanden? Wo hänge ich? Was ändere ich nächste Woche?** Für diese Sichtbarkeit lohnt sich ein Lernboard in GitHub ³⁴ Projects oder notfalls eine einfache To-do-Liste. GitHub Projects ist dafür gut geeignet, weil es Aufgaben, Fortschritt und Roadmaps sichtbar macht. ³⁵

Damit du nicht die Lust verlierst, müssen deine Übungsprojekte aus deinem eigenen Kontext kommen. Gute Mini-Projekte wären etwa:

- eine SQL-Datenbank für Bewerberstatus, Stellen, Interviews und Absagen,
- eine Python-Auswertung für Schulungsteilnahmen oder Fehlzeiten aus CSV-Dateien,
- ein kleines Reporting-Skript, das Personalzahlen pro Bereich aggregiert,
- ein GitHub-Board, in dem du Anforderungen, offene Fragen und technische Risiken dokumentierst.

Das ist nicht nur motivierender, sondern schärft auch die Brücke zwischen HR-Denke und IT-Denke. Du lernst dann nicht „Programmierung im luftleeren Raum“, sondern Datenmodelle, Logik und

Zusammenarbeit an Problemen, die dir vertraut sind. Diese Art von aktiver, problemorientierter Arbeit passt deutlich besser zu nachhaltigem Lernen als reines Video-Bingen. ³⁶

Und noch etwas, ohne Schönreden: Frust kommt auf jeden Fall. Entscheidend ist, wie du ihn einordnest. APA-Material zu Growth Mindset verweist darauf, dass solche Denkweisen Motivation und Lernen positiv beeinflussen können. Für dich heißt das praktisch: **Fehler sind kein Zeichen, dass du ungeeignet bist; sie sind der Stoff, aus dem technische Kompetenz entsteht.** Wenn du 20 Minuten festhängst, wechselst du nicht die komplette Plattform, sondern nimmst gezielt eine Hilfsquelle: Doku, eine einzelne YouTube-Erklärung oder ein Kursforum. Nicht fünf Plattformen parallel. ³⁷

Hardwareeinschätzung

Der **Lenovo-Laptop** ist dein Hauptgerät. Nicht weil er stark ist, sondern weil er für dein Vorhaben stark genug ist. Linux Mint nennt 2 GB RAM und 20 GB Speicher als Mindestanforderung; Xfce wird von der Installationsdokumentation ausdrücklich als leichtgewichtig beschrieben, das auf älteren Rechnern schneller laufen kann und sehr sparsam mit Ressourcen umgeht. Dein Gerät liegt mit 8 GB RAM und SSD darüber, also reicht es für Browser, Python, SQLite, Git und einen leichten Editor locker aus. ³⁸

Wenn auf dem Gerät noch **Windows 10** läuft, ist das im April 2026 keine saubere Dauerlösung mehr: Laut Microsoft endete der Support am **14. Oktober 2025**; seitdem gibt es keine normalen Sicherheitsupdates mehr. Windows 11 setzt wiederum unter anderem einen zugelassenen Prozessor voraus. Für ältere Hardware ist deshalb ein leichtes Linux die pragmatischste, günstigste und langfristig sauberere Lösung. Linux Mint 22 ist zudem ein LTS-Release mit Support bis 2029. ³⁹

Der **Raspberry Pi 3 B+** ist für dein Vorhaben **nicht** als Haupt-Lernrechner zu empfehlen. Der Hersteller nennt 1.4 GHz Quad-Core und 1 GB LPDDR2 RAM; außerdem braucht der Pi als normales System Boot-Medien, Stromversorgung und oft zusätzliche Peripherie. Für Headless-Linux, SSH, Dateisysteme, einfache Python-Skripte oder kleine SQLite-Experimente ist das okay. Für mehrere Browser-Tabs, moderne Videokurse, komfortables Coden und gleichzeitiges Lernen ist das aber unnötig zäh. Meine klare Wertung lautet daher: **brauchbar als Nebenprojekt, ungeeignet als primäre Lernmaschine.**

⁴⁰

Empfohlene Startkonfiguration

Meine pragmatische Empfehlung ist diese Konfiguration: **Linux Mint Xfce auf dem Lenovo-Laptop**, dazu **SQLite + DB Browser for SQLite, Python, Git** und ein einfacher Editor. DB Browser for SQLite ist ausdrücklich als visuelles Open-Source-Werkzeug für SQLite-Datenbanken gedacht und bietet eine tabellenartige Oberfläche plus SQL-Abfragefenster. Die offizielle SQLite-CLI ist zusätzlich die sauberste Referenz, wenn du später SQL auch ohne GUI verstehen willst. Diese Kombination ist leicht, kostenlos und genau tief genug für deinen Einstieg. ⁴¹

Wenn der Laptop unter Linux flüssig läuft, kannst du später noch einen üblichen Editor wie VS Code ergänzen; die Stack-Overflow-Daten zeigen Visual Studio Code weiterhin als dominierende Entwicklungsumgebung. Aber installiere am Anfang nur das Nötigste. Du lernst Informatik, nicht IDE-Verwaltung. ⁴²

Die Ersteinrichtung würde ich so angehen:

1. **Alle Daten sichern** und den Laptop wirklich als Lernmaschine freimachen, wenn du ihn neu aufsetzt.

2. **Linux Mint Xfce 22.x herunterladen** und die ISO prüfen. Das offizielle Installationshandbuch empfiehlt ausdrücklich die Verifikation von Integrität und Authentizität der ISO. ⁴³
3. **Mit einem USB-Stick ein bootfähiges Medium erstellen**, idealerweise per Etcher. Linux Mint beschreibt USB ausdrücklich als einfachsten Installationsweg. ⁴⁴
4. **Vom USB-Stick booten** und in die Live-Session starten. Dort testest du sofort WLAN, Tastatur, Touchpad, Ton und Anzeige, bevor du wirklich installierst. ⁴⁵
5. **Saubere Neuinstallation durchführen**: Sprache wählen, Internet verbinden, Multimedia-Codecs mitnehmen und die Platte löschen, wenn der Rechner wirklich nur noch dafür gedacht ist. ⁴⁶
6. **Nach dem ersten Start alle Updates einspielen** und das System einmal sauber neu starten. Linux Mint positioniert sich ausdrücklich als „out of the box“-System mit zentralen Updates und wenig Wartungsaufwand. ⁴⁷
7. **Driver Manager öffnen** und empfohlene Treiber installieren. Das nennt die Mint-Dokumentation selbst als eine der ersten Aufgaben nach der Installation. ⁴⁸
8. **Nur die Kernwerkzeuge installieren**: DB Browser for SQLite, Git und einen einfachen Editor; Python installierst du nur dann zusätzlich, wenn es nicht schon vorhanden ist. Für SQL startest du wahlweise mit dem GUI-Tool DB Browser und ergänzend mit der SQLite-CLI. ⁴⁹
9. **Arbeitsstruktur anlegen**: einen Ordner `sql`, einen Ordner `python`, einen Ordner `notizen`. Dann direkt den ersten funktionierenden Test machen: eine kleine SQLite-Datei anlegen, eine Tabelle erstellen, drei Datensätze einfügen, eine Abfrage laufen lassen. ⁵⁰
10. **Ein erstes Lern-Repository in GitHub anlegen** und deine Aufgaben als Issues oder in einem Board verwalten. So lernst du von Anfang an nicht nur Code und Daten, sondern auch die Arbeitsweise von Softwareteams. ²⁴

Wenn du es maximal pragmatisch willst, dann ist das hier die Reihenfolge, mit der du morgen anfangen solltest: **Laptop aufsetzen, SQL im Browser lernen, SQLite lokal ausprobieren, danach Python dazunehmen, Web-Grundlagen erst später, Java und Angular vorerst ignorieren**. Genau so minimierst du Reibung und maximierst die Chance, dass du wirklich dranbleibst. ⁵¹

¹ ³ ⁴ ²⁶ ⁵¹ <https://sqlbolt.com/>
<https://sqlbolt.com/>

² ³⁸ <https://linuxmint.com/faq.php>
<https://linuxmint.com/faq.php>

⁵ ²⁷ <https://cs50.harvard.edu/sql>
<https://cs50.harvard.edu/sql>

⁶ ²⁸ <https://cs50.harvard.edu/python>
<https://cs50.harvard.edu/python>

⁷ <https://docs.python.org/3/tutorial/index.html>
<https://docs.python.org/3/tutorial/index.html>

⁸ <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>
<https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>

⁹ ¹⁴ <https://angular.dev/tutorials/learn-angular>
<https://angular.dev/tutorials/learn-angular>

¹⁰ ²⁴ ³⁰ <https://learn.microsoft.com/en-us/training/modules/introduction-to-github/>
<https://learn.microsoft.com/en-us/training/modules/introduction-to-github/>

- 11 47 <https://linuxmint.com/>
<https://linuxmint.com/>
- 12 21 <https://git-scm.com/book/en/v2>
<https://git-scm.com/book/en/v2>
- 13 45 <https://linuxmint-installation-guide.readthedocs.io/en/latest/boot.html>
<https://linuxmint-installation-guide.readthedocs.io/en/latest/boot.html>
- 15 16 <https://www.freecodecamp.org/news/about/>
<https://www.freecodecamp.org/news/about/>
- 17 <https://www.linkedin.com/help/learning/answer/a700791>
<https://www.linkedin.com/help/learning/answer/a700791>
- 18 <https://blog.coursera.org/introducing-courseras-new-course-preview-experience/>
<https://blog.coursera.org/introducing-courseras-new-course-preview-experience/>
- 19 <https://support.udemy.com/hc/en-us/articles/229605368-Instructors-Udemy-s-Pricing-Tiers-For-Courses>
<https://support.udemy.com/hc/en-us/articles/229605368-Instructors-Udemy-s-Pricing-Tiers-For-Courses>
- 20 <https://support.google.com/youtube/answer/57792?co=GENIE.Platform%3DAndroid&hl=en>
<https://support.google.com/youtube/answer/57792?co=GENIE.Platform%3DAndroid&hl=en>
- 22 25 <https://www.khanacademy.org/computing/computer-programming/sql>
<https://www.khanacademy.org/computing/computer-programming/sql>
- 23 29 https://developer.mozilla.org/en-US/docs/Learn_web_development
https://developer.mozilla.org/en-US/docs/Learn_web_development
- 31 34 <https://cs50.harvard.edu/x/>
<https://cs50.harvard.edu/x/>
- 32 36 <https://learningcenter.unc.edu/tips-and-tools/studying-101-study-smarter-not-harder/>
<https://learningcenter.unc.edu/tips-and-tools/studying-101-study-smarter-not-harder/>
- 33 <https://learningcenter.unc.edu/tips-and-tools/motivation/>
<https://learningcenter.unc.edu/tips-and-tools/motivation/>
- 35 <https://learningcenter.unc.edu/tips-and-tools/metacognitive-study-strategies/>
<https://learningcenter.unc.edu/tips-and-tools/metacognitive-study-strategies/>
- 37 <https://www.apa.org/ed/precollege/psychology-teacher-network/introductory-psychology/college-growth-mindset>
<https://www.apa.org/ed/precollege/psychology-teacher-network/introductory-psychology/college-growth-mindset>
- 39 <https://www.microsoft.com/de-de/windows/end-of-support>
<https://www.microsoft.com/de-de/windows/end-of-support>
- 40 <https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>
<https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>
- 41 <https://linuxmint-installation-guide.readthedocs.io/en/latest/choose.html>
<https://linuxmint-installation-guide.readthedocs.io/en/latest/choose.html>
- 42 <https://survey.stackoverflow.co/2025/>
<https://survey.stackoverflow.co/2025/>
- 43 <https://linuxmint-installation-guide.readthedocs.io/>
<https://linuxmint-installation-guide.readthedocs.io/>

44 <https://linuxmint-installation-guide.readthedocs.io/en/latest/burn.html>

<https://linuxmint-installation-guide.readthedocs.io/en/latest/burn.html>

46 <https://linuxmint-installation-guide.readthedocs.io/en/latest/install.html>

<https://linuxmint-installation-guide.readthedocs.io/en/latest/install.html>

48 <https://linuxmint-installation-guide.readthedocs.io/en/latest/drivers.html>

<https://linuxmint-installation-guide.readthedocs.io/en/latest/drivers.html>

49 <https://sqlitebrowser.org/>

<https://sqlitebrowser.org/>

50 <https://sqlite.org/cli.html>

<https://sqlite.org/cli.html>